# Proof-of-Inference: End-to-End Emission and On-Chain Enforcement

The activation-trace commitment that makes "no real compute, no mint" a theorem, not a hope

Zoo Labs Foundation
Hanzo AI
Lux Network

**Abstract**

A companion proof established that the Freivalds matrix-product verifier over $\mathbb{F}_p$, $p = 2^{61}-1$, catches a fabricated product $C \neq A \cdot B$ with probability $\geq 1 - 1/p$ per challenge vector, with *zero* false rejection over an exact integer accumulator. That primitive was correct but *unwired*: nothing fed it operands from a real forward pass, so an adversary could mint by committing to an output it never computed. This paper closes that gap. We define an *activation-trace transcript* — a keccak Merkle commitment over the exact-integer operands of each proof-bearing matmul of a forward pass — and a permissionless *challenger* that opens a beacon-selected matmul and Freivalds-checks it. We prove (Theorem 3) that a prover who returns any output not produced by the committed computation is caught except with probability $\leq (1/p)^k$, and that an honest prover is *never* slashable (Theorem 5). We give a proof-bearing forward (the `ProvableLinear` layer) whose returned value *is* the committed exact-integer result, eliminating the float-shadow unsoundness; an on-chain witness (`ComputeWitnessLib`) and a native precompile (`aivmbridge` selector `0x12000000`) that run the check in the EVM with no off-chain trust; and a slash gate (`OptimisticEvidence`) that pays a bond to a challenger *only* on a verified fraud. Every result is implemented and tested across four languages (Rust prover, Go verifier, Solidity contract, Go precompile) with byte-identical commitments and a pinned cross-language challenge vector; §7 tabulates the suite, including an adversarial red-team battery that exercises each attack the theorems forbid.

## 1 Setting and the gap this closes

An LLM forward pass is, to ∼95% of its arithmetic, a sequence of matrix multiplications $C^{(\ell)} = A^{(\ell)}B^{(\ell)}$, $\ell = 1, \ldots, L$. Proof-of-Inference binds a coin reward to having *performed* that arithmetic. The verifier is Freivalds' check [1]: for a random $r$, test $A(Br) \stackrel{?}{=} Cr$ over $\mathbb{F}_p$ in $O(tk + kn + tn)$ — an order cheaper than recomputing $C$ — catching $C \neq AB$ with probability $\geq 1 - 1/p$.

We work over the Mersenne prime $p = 2^{61} - 1$ on the *exact integer accumulator* of the int8 matmul path ($i8 \times i8 \to i64$, whole-$K$, no cross-block float reduction). Integer arithmetic is bit-identical across CPU/GPU/backends, giving zero false rejection. The companion paper proves the single-product soundness; we take it as Lemma 1 and build the system that feeds it.

**Lemma 1** (Single-product soundness, companion)**.** *For integer matrices $A \in \mathbb{Z}^{t\times k}, B \in \mathbb{Z}^{k\times n}, C \in \mathbb{Z}^{t\times n}$ reduced into $\mathbb{F}_p$ entrywise, and $k$ challenge vectors $r_1, \ldots, r_k$ drawn uniformly and independently from $\mathbb{F}_p^n$: if $C = AB$ then $A(Br_j) = Cr_j$ for all $j$ (zero false reject); if $C \neq AB$ then $\Pr[\,\forall j : A(Br_j) = Cr_j\,] \leq (1/p)^k$.*

**The gap.** Lemma 1 is about *operands*. The unsoundness the audit found was that nothing in the deployed system ever produced operands from a real model: the verifier had zero callers, and the mint gate accepted any non-empty "witness". So a prover committed an output hash with no computation behind it and minted. Everything below is the missing wire.

## 2 The activation-trace transcript

**Definition 1** (Matmul leaf)**.** *For a matmul with exact integer operands $(A, B, C)$, its leaf is*

$$\mathsf{leaf}(A, B, C) \triangleq \mathsf{keccak}\big(\,\texttt{DOM} \,\|\, \mathsf{enc}(A) \,\|\, \mathsf{enc}(B) \,\|\, \mathsf{enc}(C)\,\big),$$

*where* $\mathsf{enc}(M) = \mathrm{BE}_{32}(\mathit{rows}) \,\|\, \mathrm{BE}_{32}(\mathit{cols}) \,\|\, \big(\mathrm{BE}_{64}(M_{ij})\big)_{ij}$ *is the canonical row-major serialization and* $\texttt{DOM} =$ `''hanzo/poi/matmul-leaf/v1''`.

**Definition 2** (Transcript and root)**.** *A forward pass commits its proof-bearing matmuls $(A^{(\ell)}, B^{(\ell)}, C^{(\ell)})_{\ell=1}^{L}$ as the leaf vector $(\mathsf{leaf}_\ell)_\ell$. The* transcript root $\rho$ *is the keccak binary Merkle root over those leaves (odd levels duplicate the last node; the index fold is* $\mathsf{even} \to \mathsf{keccak}(\mathit{node} \| \mathit{sib})$, $\mathsf{odd} \to \mathsf{keccak}(\mathit{sib} \| \mathit{node})$*, identical to the deployed* `_verifyMerkle`*). The prover posts $\rho$ on-chain alongside its claimed output.*

**Definition 3** (Opening)**.** *To* open *index $\ell$, the prover reveals $(A^{(\ell)}, B^{(\ell)}, C^{(\ell)})$ and the Merkle path $\pi_\ell$. The opening is* included *iff folding $\mathsf{leaf}(A^{(\ell)}, B^{(\ell)}, C^{(\ell)})$ with $\pi_\ell$ under the index reproduces $\rho$.*

**Lemma 2** (Binding)**.** *Under collision resistance of* keccak*, an opening at index $\ell$ that is included under $\rho$ reveals exactly the operands the prover committed at $\ell$: producing an included opening with any $(A', B', C') \neq (A^{(\ell)}, B^{(\ell)}, C^{(\ell)})$ requires a keccak collision.*

*Proof.* Inclusion forces $\mathsf{leaf}(A', B', C')$ to equal the committed leaf at $\ell$ (the Merkle path and index are fixed by $\rho$ and a second pre-image at any internal node is a collision). Equal leaves with distinct operands is a collision of keccak on the encoded operand string, since enc is injective (fixed-width, length-prefixed). □ □

## 3 The challenger and end-to-end soundness

**Definition 4** (Beacon-bound challenge)**.** *Given a public, post-commitment beacon $\beta$ (§5), the challenge for index $\ell$ is $r_{j,i} = \mathrm{BE}_{64}\big(\mathsf{keccak}(\sigma \,\|\, \mathrm{BE}_{32}(j) \,\|\, \mathrm{BE}_{64}(i))[0{:}8]\big) \bmod p$ with seed $\sigma = \mathsf{keccak}(\beta \| \rho \| \mathrm{BE}_{64}(\ell))$, for $j < k$, $i < n$. Modeling* keccak *as a random oracle, the $r_j$ are uniform and independent in $\mathbb{F}_p^n$ and are fixed only* after *$\rho$ is committed.*

**Definition 5** (VerifyOpening, ProvesFraud)**.** $\mathsf{VerifyOpening}(\rho, \beta, \mathit{op}) = \mathsf{included}(\mathit{op}, \rho) \wedge \mathsf{Freivalds}_k(A, B, C)$ *with the beacon-bound challenge. The fraud predicate is its dual on the correctness bit:* $\mathsf{ProvesFraud} = \mathsf{included} \wedge \neg\mathsf{Freivalds}_k$.

**Theorem 3** (A fabricated matmul is provable)**.** *Suppose the prover's committed transcript contains, at some index $\ell$, a matmul with $C^{(\ell)} \neq A^{(\ell)}B^{(\ell)}$. Then a challenger that opens $\ell$ produces a witness for which* $\mathsf{ProvesFraud} = \mathsf{true}$ *except with probability $\leq (1/p)^k$ over the beacon. With $k = 2$ and $p = 2^{61} - 1$ this failure probability is $\leq 2^{-122}$.*

*Proof.* The honest opening of $\ell$ is included (Lemma 2 gives inclusion for the committed operands), so $\mathsf{included} = \mathsf{true}$. Because $C^{(\ell)} \neq A^{(\ell)}B^{(\ell)}$, Lemma 1 gives $\Pr[\mathsf{Freivalds}_k = \mathsf{true}] \leq (1/p)^k$, hence $\neg\mathsf{Freivalds}_k$ holds except with that probability, and $\mathsf{ProvesFraud} = \mathsf{true}$. The beacon is drawn after $\rho$ is fixed, so the prover cannot have chosen $C^{(\ell)}$ to satisfy the specific challenge (the pre-fit attack of §5). □ □

**Corollary 4** (No-compute ⇒ provable fraud)**.** *A permissionless watcher that re-executes the committed trace off-chain finds every index where* $C^{(\ell)} \neq A^{(\ell)}B^{(\ell)}$ deterministically *(it recomputes* $A^{(\ell)}B^{(\ell)}$ *in the clear) and opens one. Thus any forward pass whose committed output was not actually computed contains at least one such* $\ell$ *and is slashable except with probability* $\leq (1/p)^k$*.*

**Theorem 5** (An honest prover is never slashable)**.** *If every committed matmul satisfies* $C^{(\ell)} = A^{(\ell)}B^{(\ell)}$*, then for every index* $\ell$*, every beacon* $\beta$*, and every (even adversarially chosen) opening submitted by a griefer,* $\mathsf{ProvesFraud} = \mathsf{false}$*.*

*Proof.* A griefer's opening either (i) reveals the committed operands, in which case $\mathsf{Freivalds}_k = \mathsf{true}$ by zero-false-reject (Lemma 1), so $\neg\mathsf{Freivalds}_k = \mathsf{false}$; or (ii) reveals different operands, in which case $\mathsf{included} = \mathsf{false}$ except for a keccak collision (Lemma 2). Either way $\mathsf{ProvesFraud} = \mathsf{included} \wedge \neg\mathsf{Freivalds}_k = \mathsf{false}$. □ □

## 4 Sound emission: the output is the committed value

Theorems 3–5 bind *committed matmuls*. They are only meaningful if the value the model *returns* is determined by those matmuls. The `ProvableLinear` layer makes this hold by construction.

**Definition 6** (Proof-bearing linear)**.** *For int8 weights* $W \in [-127, 127]^{o\times m}$ *with public per-row dequant scales* $s^W \in \mathbb{R}^o$*, the layer on input* $x \in \mathbb{R}^{t\times m}$ *computes the per-row symmetric int8 quantization* $(\bar{x}, s^x) = \mathsf{quant}(x)$ *(deterministic:* $s^x_r = \max_j |x_{rj}|/127$*,* $\bar{x}_{rj} = \mathrm{round}(x_{rj}/s^x_r)$*), forms* $C = \bar{x}\, W^\top$ *on the exact* $i64$ *accumulator, commits* $(\bar{x}, W^\top, C)$*, and returns* $y_{ro} = C_{ro}\, s^x_r\, s^W_o$*.*

**Proposition 6** (Output binding)**.** *The returned* $y$ *is a fixed, public function* $\mathsf{dequant}(\cdot)$ *of the committed* $C$ *and the committed/public scales. Hence any party can recompute* $y$ *from the opened final-layer* $C$ *and reject a prover that claims a different output; and two honest executions emit byte-identical transcripts.*

*Proof.* quant and dequant are deterministic given $x$ and the public scales, and $C$ is the exact integer product (no floating-point reduction), so it is reproducible across backends. $y = \mathsf{dequant}(C)$ is then determined by the committed $C$; equality of two honest runs follows from determinism of every step. □ □

**Remark 1** (Why exact integer, not a float shadow)**.** *A kernel that accumulates int8 block dots in* `f32` *(the common fast path) is non-associative and not bit-identical across hardware; committing its* $i64$ *"shadow" would commit a computation different from the returned value, breaking Proposition 6.* `ProvableLinear` *therefore* is *the computation: the returned value is dequantized directly from the committed exact* $C$*. The throughput cost of the exact accumulator is the price of a reproducible proof and is paid only on proof-bearing inferences.*

## 5 The post-commitment beacon

Soundness (Theorem 3) requires the challenge to be unpredictable when the prover chooses $C$. If the seed depended only on $\rho$ (which the prover knows), a prover could fabricate $C' = AB + \Delta$ with $\Delta r = 0$ for the resulting $r$ and pass. We therefore bind the beacon to a value fixed *after* the prover's commitment is mined: $\beta = \mathsf{keccak}(\mathsf{blockhash}(b_0) \,\|\, \mathsf{id})$ where $b_0$ is the commitment's block. A transaction cannot contain its own block's hash, so a prover choosing $C$ before signing cannot predict $\beta$; blockhash is available for 256 blocks, bounding the challenge window. The pre-fit attack is exactly attack 6 of the red-team battery (§7): a forgery tuned to a guessed $r_1$ is shown to pass $r_1$ and fail a fresh $r_2$.

## 6 On-chain enforcement, with no off-chain trust

The witness is checked in the EVM, not asserted by a watcher. Two implementations share one wire format and one field:

- **Reference (Solidity).** `ComputeWitnessLib.verifyOpening` runs Freivalds over $\mathbb{F}_p$ via `mulmod`/`addmod` plus the Merkle fold and the beacon challenge, byte-identical to the prover. Cost is $O(tk+kn+tn)$ field ops — affordable for a bounded slice; a full layer is challenged in slices.
- **Native (precompile).** `aivmbridge` at `0x0300...0004`, selector `0x12000000`, decodes the opening wire and calls the canonical Go verifier, returning $(\mathsf{included} \mid \mathsf{freivaldsOK} \ll 1)$. Gas is base + (field ops) $\cdot\, c$, priced so a 256-wide slice costs ∼1M gas and a 4096-wide layer is unaffordable, forcing bounded slices; `RequiredGas` equals the metered consumption.

**Proposition 7** (Gate soundness)**.** *The slash gate* `OptimisticEvidence.challenge` *transitions a commitment to* FRAUDULENT *and pays the bond to the caller* iff ProvesFraud *holds for an opening under the committed root and the commit-block beacon. Consequently (Theorems 3, 5) a fabricated forward pass is slashable except with probability* $\leq (1/p)^k$*, and an honest prover is never slashable.*

*Proof.* By inspection the gate calls ProvesFraud with $\beta =$ blockhash(commitBlock) and slashes exactly on its truth, within the open window (after which the commitment finalizes). The two theorems then transfer. □ □

## 7 Implementation and adversarial evidence

Every definition and theorem above is realized in code, with the commitment, the field, the Merkle fold, and the challenge derivation *byte-identical* across the prover, the watcher, the contract, and the precompile (enforced by a pinned cross-language golden vector that breaks one suite if either side drifts). The honest path is exercised on a real SwiGLU FFN forward; the adversarial path on a battery that instantiates each attack the theorems forbid.

| Component | What it realizes | Tests |
|---|---|---|
| `hanzo-engine/poi.rs` | exact matmul; keccak; Go-parity challenge derivation; Freivalds | |
| `hanzo-engine/poi_transcript.rs` | transcript, Merkle root/proof, `verify_opening` (§2–3) | |
| `hanzo-engine/poi_forward.rs` | `ProvableLinear`, `prove` (§4) | 18/18 |
| `crypto/poi` (Go) | transcript + `VerifyOpening` + opening wire codec | pass |
| `ComputeWitnessLib.sol` | on-chain Freivalds + Merkle + beacon (§6) | 5/5 |
| `OptimisticEvidence.sol` | the slash gate, `ProvesFraud`-driven (Prop. 7) | 18/18 |
| `aivmbridge` precompile | native verification, selector `0x12` | 5/5 |
| `tests/poi_adversarial.rs` | red-team battery (below) | pass |

**Red-team battery (each maps to a theorem).**

1. Fabricate the final-layer output ⇒ caught (Thm. 3).
2. Fabricate a hidden intermediate matmul ⇒ caught when opened; honest layers still verify.
3. Sparse deep cheat (one wrong entry in a 256-deep matmul) ⇒ caught.

4. Swap the output at open time ⇒ Merkle inclusion fails (Lemma 2).
5. Tamper an input at open time ⇒ inclusion fails (the leaf binds $A$).
6. Pre-fit $C$ to a *guessed* challenge $r_1$ ⇒ passes $r_1$, fails the beacon-fresh $r_2$ (§5).
7. Griefer submits a fabricated opening not under the honest root ⇒ cannot slash (Thm. 5).
8. Quantization lie (claim $y \neq \mathsf{dequant}(C)$) ⇒ detectable by public recompute (Prop. 6).
9. Determinism: two honest machines emit the byte-identical root (zero false reject).
10. Honest forward verifies under *every* beacon.

## 8 Scope: what is proven and what is assumed

- **Proven (here, on code).** Per-matmul soundness end to end (Thm. 3); no-false-slash (Thm. 5); output binding for the proof-bearing linear (Prop. 6); on-chain enforcement with native and reference verifiers (Prop. 7); cross-language byte parity.
- **Assumed.** Collision/pre-image resistance and random-oracle behavior of keccak; the field bound of Lemma 1 (companion); an honest majority of block producers for the blockhash beacon over the challenge window (a stronger VRF beacon strengthens this without changing the theorems).
- **Out of scope (named, not hidden).** The *sequential-chaining* obligation — that $A^{(\ell+1)}$ is the deterministic requantization of $\mathsf{dequant}(C^{(\ell)})$ — is checked per link by recomputation but its full-graph composition (attention, MoE routing) is the subject of the companion extensions; confidentiality of the prompt is a separate envelope and is not claimed here.

## References

[1] R. Freivalds, *Probabilistic machines can use less running time*, IFIP Congress, 1977.